

Benutzerhandbuch

Der Assembler in der Anwendung: Kommandozeile, Quelltext-Dialekte, Plattformen, Ausgabeformate, Dialekt-Umwandlung und REST-API — und die vollständige Referenz des rat-Dialekts, aller CPUs, Plattformen und Optionen

13. November 2026

Inhalt

Teil I — rat anwenden 4

1 Einführung 5

1.1 Was rat kann . . . 5
1.2 Unterstützte CPUs . . . 5
1.3 Drei Wege der Benutzung . . . 6

2 Installation und Schnellstart 7

2.1 Fertige Programme . . . 7
2.2 Downloads prüfen . . . 7
2.3 Aus den Quellen bauen . . . 8
2.4 Ein erstes Programm . . . 8

3 Die Kommandozeile 9

3.1 Assemblieren: rat build . . . 9
3.2 Listing und Label-Dateien . . . 9
3.3 Disassemblieren: rat disasm . . . 10
3.4 Umwandeln: rat convert . . . 10
3.5 Server: rat serve . . . 10

4 Quelltext-Dialekte 11

4.1 Welchen Dialekt eine Datei hat . . . 11
4.2 Dasselbe Programm in drei Dialekten . . . 11

5 Plattformen 13

5.1 Übersicht . . . 13
5.2 Plattform-Direktiven . . . 14

6 Maschine für Maschine 15

6.1 Commodore: C64, C128, VIC-20, PET, C16/Plus/4, MEGA65 . . . 15
6.2 Atari 8-Bit und Atari 2600 . . . 15
6.3 NES und SNES . . . 15
6.4 Z80-Maschinen: CPC, ZX81, Spectrum, ColecoVision, MSX, MSX2, TRS-80 . . . 16
6.5 Acorn: BBC Micro, BBC Master, Archimedes . . . 17
6.6 Apple II und Macintosh . . . 18
6.7 Amiga . . . 18
6.8 Vectrex, TI-99/4A, IMSAI 8080 . . . 18

7 Zusammenspiel mit mimic 20

8 Zwischen Dialekten umwandeln 21

9 REST-API 22

Teil II — Referenz 23

10 Der rat-Dialekt **24**

10.1 Labels und Symbole . . . 24
10.2 Zahlen, Zeichenketten, Operatoren . . . 24
10.3 Daten . . . 25
10.4 Programmzähler und Speicher . . . 25
10.5 Ablaufsteuerung und Wiederverwendung . . . 25
10.6 Direktiven für CPU, Plattform und Ziel . . . 26

11 Operandenschreibweise je CPU **27**

12 CPUs **28**

13 Plattformen **29**

13.1 Plattform-Direktiven und -Funktionen . . . 30

14 Ausgabeformate **32**

15 Referenz der Kommandozeile **33**

15.1 rat build QUELLE . . . 33
15.2 rat disasm DATEI . . . 33
15.3 rat convert DATEI . . . 33
15.4 rat serve . . . 33

16 Referenz der REST-API **34**

16.1 POST /assemble . . . 34
16.2 POST /disassemble . . . 34
16.3 POST /convert . . . 34
16.4 GET /cpus, /platforms, /formats, /health . . . 35

17 Bekannte Lücken **36**

18 Wie rat arbeitet **37**

18.1 Durchläufe . . . 37
18.2 Relozierung beim Amiga . . . 37
18.3 Tests . . . 37
18.4 Dieses Handbuch bauen . . . 37

Teil I — rat anwenden

Installation, Kommandozeile, Quelltext-Dialekte, Plattformen, Beispiele für die Maschinen, Umwandlung und REST-API

1 Einführung

rat assembliert Text. rat ist ein in Rust geschriebener Assembler für viele CPUs und mehrere Dialekte, gebaut für jede CPU, die der Emulator mimic ausführt. Ein Programm deckt die 6502-Familie, den Z80, den 68000 und die anderen Prozessoren der klassischen Heimcomputer und Konsolen ab und schreibt die Dateiformate, die diese Maschinen laden: PRG, D64, Modul-ROMs, Band- und Disketten-Images.

Dieses Handbuch beschreibt die Anwendung: Installation, Programme auf der Kommandozeile bauen, Dialekt und Plattform wählen, die Ausgabeformate schreiben, Quelltexte zwischen den Dialekten umwandeln und die REST-API benutzen. Teil II ist die Referenz des rat-Dialekts, der CPUs, der Plattformen, der Formate und aller Optionen.

1.1 Was rat kann

- **Assembliert** ACME-, KickAssembler- und rat-Quelltexte, auch gemischt in einem Projekt.
- **Kennt 16 CPUs** mit ihrer eigenen Operandenschreibweise (MOS, Zilog, Intel, Motorola, Sony, TI, Acorn).
- **Kennt 30 Plattformen**: jede legt CPU, Startadresse, Textkodierung, Ausgabeformate und eigene Direktiven fest.
- **Schreibt 19 Ausgabeformate**, vom rohen Binärabbild bis zum D64-Image oder zum Amiga-Hunk-Programm.
- **Disassembliert** Binärdateien jeder CPU in rat-Syntax.
- **Wandelt** Quelltexte zwischen den drei Dialekten um.
- **Bietet eine REST-API** mit denselben Funktionen, für Editoren und Web-Werkzeuge.

1.2 Unterstützte CPUs

CPU	Kennungen (`--cpu`, `.cpu`)	Byte-Reihenfolge	Verwendet von
MOS-6502-Familie	`6502` (mit undokumentierten Opcodes), `6502ni`, `6510`, `2a03`, `6507`	little	C64, C128, VIC-20, PET, C16/Plus/4, Atari 8-Bit, Atari 2600, NES, Apple II, BBC Micro
CMOS 65C02	`65c02` (ohne Rockwell-Bitbefehle)	little	Apple IIe enhanced, BBC Master (65SC12)
MEGA65 45GS02	`45gs02` (4510/65CE02 mit Q-Befehlen, flachen Zeigern, langen Sprüngen)	little	MEGA65
WDC 65C816	`65816`	little	SNES
Sony SPC700	`spc700`	little	SNES-Ton
Zilog Z80 (mit undokumentierten Befehlen)	`z80`	little	Amstrad CPC, ZX81, ZX Spectrum, ColecoVision, MSX, MSX2, TRS-80, C128
Intel 8080	`8080` (Intel-Mnemonics)	little	IMSAI 8080
TI TMS9900	`tms9900`	big	TI-99/4A
ARM2	`arm2` (26 Bit, Acorn-Mnemonics)	little	Acorn Archimedes
Motorola 6809	`6809`	big	Vectrex
Motorola 68000 / 68020	`68000`, `68020`	big	Amiga 500 bis 1200, Macintosh

Die Kodiertabellen stammen aus mimics CPU-Kernen. Die Tests prüfen jeden Opcode gegen mimics Disassembler (rund 180 000 Befehle): was rat assembliert, ist das, was mimic ausführt.

1.3 Drei Wege der Benutzung

Weg	Wie
Kommandozeile	`rat build`, `rat disasm`, `rat convert` und die Listenbefehle `rat cpus`, `rat platforms`, `rat formats`.
REST-API	`rat serve` startet einen HTTP-Server mit `/assemble`, `/disassemble` und `/convert`; OpenAPI-Beschreibung und Swagger UI inklusive.
Bibliothek	Die Rust-Crates `rat-asm`, `rat-cpu`, `rat-platform` und `rat-output` lassen sich aus anderen Rust-Programmen benutzen.

2 Installation und Schnellstart

2.1 Fertige Programme

Fertige Programme für Linux, macOS und Windows hängen an jedem GitHub-Release (github.com/guidobonerz/rat/releases). Das Archiv für das eigene System herunterladen und entpacken; `rat` ist eine einzelne Datei und braucht keine Installation.

Linux

```
tar -xzf rat-v0.1.0-x86_64-unknown-linux-gnu.tar.gz
cd rat-v0.1.0-x86_64-unknown-linux-gnu
chmod +x rat                      # nur nötig, wenn das Ausführ-Bit verloren ging
sudo mv rat /usr/local/bin/       # oder ~/.local/bin, wenn das im PATH liegt
rat --help
```

Auf ARM64-Rechnern wie einem Raspberry Pi 4/5 mit 64-Bit-System das Archiv `aarch64-unknown-linux-gnu` nehmen.

macOS

`aarch64-apple-darwin` für Apple Silicon (M1 und neuer), `x86_64-apple-darwin` für Intel-Macs.

```
tar -xzf rat-v0.1.0-aarch64-apple-darwin.tar.gz
cd rat-v0.1.0-aarch64-apple-darwin
chmod +x rat
xattr -d com.apple.quarantine rat # das Programm ist weder signiert noch notarisiert
sudo mv rat /usr/local/bin/
rat --help
```

Ohne den `xattr`-Schritt blockiert macOS das Programm mit der Meldung, der Entwickler könne nicht überprüft werden. Alternativ: einmal starten, dann unter *Systemeinstellungen → Datenschutz & Sicherheit → Dennoch öffnen* erlauben.

Windows

Das Zip entpacken, zum Beispiel nach `C:\Tools\rat`, und den Ordner in den `PATH` aufnehmen, damit `rat` in jedem Terminal funktioniert:

```
setx PATH "%PATH%;C:\Tools\rat"
```

Ein neues Terminal öffnen und `rat --help` aufrufen. Warnt SmartScreen vor einer unbekannten App: *Weitere Informationen → Trotzdem ausführen*. Hat Windows die heruntergeladene Datei blockiert, hilft in der PowerShell:

```
Unblock-File C:\Tools\rat\rat.exe
```

> **Hinweis.** Die Programme für Linux und Windows sind mit UPX gepackt. Manche Virenscanner stufen UPX-gepackte Dateien als verdächtig ein. Landet `rat.exe` in der Quarantäne: gegen `SHA256SUMS.txt` prüfen und eine Ausnahme anlegen, oder aus den Quellen bauen.

2.2 Downloads prüfen

Jedes Release enthält `SHA256SUMS.txt`:

```
sha256sum -c SHA256SUMS.txt --ignore-missing          # Linux
shasum -a 256 -c SHA256SUMS.txt --ignore-missing      # macOS
Get-FileHash rat-v0.1.0-x86_64-pc-windows-msvc.zip    # Windows, von Hand vergleichen
```

2.3 Aus den Quellen bauen

- **Rust** (stabile Toolchain) mit `cargo` — Installation über rustup.rs.
- Das Repository **mimic** neben rat ausgecheckt, in einem Ordner namens `mimic`: die Tests benutzen dessen CPU-Kerne und Medienleser.

```
cargo install --path crates/rat-cli       # legt rat nach ~/.cargo/bin
cargo build --release -p rat-cli          # oder: target/release/rat
```

2.4 Ein erstes Programm

Als `hello.asm` speichern (KickAssembler-Dialekt, die Plattform ist dann der C64):

```
BasicUpstart2(start)

        *= $0810 "Main"
start:  ldx #0
!loop:  lda text,x
        beq !done+
        jsr $ffd2
        inx
        bne !loop-
!done:  rts

text:   .encoding "petscii_mixed"
        .text "hello world"
        .byte 13, 0
```

Assemblieren und das Ergebnis in mimic starten:

```
rat build hello.asm                       # -> hello.prg
mimic-desktop --machine c64 --prg hello.prg
```

Dasselbe im rat-Dialekt für eine andere Maschine, hier ein Band für den ZX Spectrum:

```
; hello.rat
.platform spectrum
.tap name="HELLO", exec=start
.org $8000
start:  ld a, 2
        call $1601          ; CHAN-OPEN (Bildschirm)
        ld hl, msg
loop:   ld a, (hl)
        or a
        ret z
        rst $10             ; Zeichen ausgeben
        inc hl
        jr loop
msg:    .text "Hello Spectrum!", 13
        .byte 0
```

```
rat build hello.rat                       # -> hello.tap, mit BASIC-Lader
```

3 Die Kommandozeile

`rat` hat einen Befehl je Aufgabe. `rat --help` und `rat <Befehl> --help` zeigen die Optionen.

Befehl	Zweck
`rat build QUELLE`	Einen Quelltext assemblieren.
`rat disasm DATEI --cpu CPU`	Eine Binärdatei in rat-Syntax disassemblieren.
`rat convert DATEI --to DIALEKT`	Einen Quelltext in einen anderen Dialekt umwandeln.
`rat serve`	Den REST-API-Server starten.
`rat cpus`	Die CPUs mit Aliasnamen, Byte-Reihenfolge und Adressbreite auflisten.
`rat platforms`	Die Plattformen mit CPU, Formaten, Direktiven und Funktionen auflisten.
`rat formats`	Die Ausgabeformate mit Endung und Plattformen auflisten.

3.1 Assemblieren: rat build

```
rat build main.asm                      # Dialekt aus der Endung, c64 -> main.prg
rat build game.rat -p nes -o game.nes
rat build demo.a -D DEBUG=1 -I lib --listing demo.lst --vice-labels demo.vs
rat build intro.asm -f d64              # D64 mit einem PRG
rat build cart.rat -p msx2 -f bin       # Modul-ROM für den MSX2
```

- Der **Dialekt** folgt der Dateiendung (`.a` ACME, `.asm` KickAssembler, `.rat` rat), solange weder `--syntax` noch eine `.syntax`-Zeile etwas anderes sagt.
- Die **Plattform** kommt von `-p`, von einer `.platform`-Zeile im Quelltext oder vom Dialekt: KickAssembler- und ACME-Quelltexte beginnen auf `c64`, rat-Quelltexte auf `generic`.
- Das **Format** ist das erste Format der Plattform, solange weder `-f` noch `.output` ein anderes nennt.
- Die **Ausgabedatei** heißt wie der Quelltext, mit der Endung des Formats, solange `-o` fehlt.
- `-D NAME` oder `-D NAME=WERT` definiert ein Symbol (KickAssembler-Quelltexte sehen es auch in `cmdLineVars`); `-I ORDNER` fügt ein Include-Verzeichnis hinzu. Beides ist mehrfach erlaubt.

Fehler erscheinen im Stil des Rust-Compilers, mit Datei, Zeile und der markierten Quellzeile. Schlägt der Bau fehl, ist der Exit-Code ungleich null.

3.2 Listing und Label-Dateien

Option	Schreibt
`--listing DATEI`	Ein Listing: Adresse, Bytes, Zeilennummer und Quellzeile.
`--vice-labels DATEI`	VICE-Labels (`al C:xxxx .name`), die mimics Debugger und VICE direkt laden; `.break` und `.watch` im Quelltext werden zu Haltepunkten.
`--sym DATEI`	Eine `.sym`-Datei im KickAssembler-Format.
`--labels DATEI`	Eine Label-Liste im ACME-Format.
`--json`	Eine Zusammenfassung von Ausgaben, Symbolen, Meldungen und Quellzeilen als JSON auf der Standardausgabe, für Build-Skripte und Editoren.

Ein Listing sieht so aus:

```
004000  c9                      1  start: ret
004001  00 40                   2         .word start
```

3.3 Disassemblieren: rat disasm

```
rat disasm game.prg --cpu 6510          # die PRG-Ladeadresse wird beachtet
rat disasm rom.bin --cpu 6809 --org $e000
rat disasm code.bin --cpu 65816 --a16 --i16
```

Die Ausgabe ist Quelltext im rat-Dialekt, der wieder dieselben Bytes ergibt. `--org` setzt die Ladeadresse, `--skip N` überspringt einen Vorspann, `--prg` behandelt die Datei unabhängig vom Namen als PRG. Beim 65816 legen `--a16` und `--i16` die Breite von Akkumulator und Indexregistern fest.

3.4 Umwandeln: rat convert

```
rat convert old.a --to kick -o new.asm   # ACME -> KickAssembler
rat convert demo.asm --to rat            # auf die Standardausgabe
```

Was dabei übernommen wird, steht im Kapitel *Zwischen Dialekten umwandeln*.

3.5 Server: rat serve

```
rat serve                                # 127.0.0.1:8080
rat serve --bind 0.0.0.0:9000
```

Siehe das Kapitel *REST-API*.

4 Quelltext-Dialekte

rat liest drei Dialekte und kann zwischen ihnen umwandeln.

Dialekt	Endung	Merkmale
ACME	`.a`	`!`-Pseudobefehle, Labels ohne Doppelpunkt, `.lokale` und `@cheap`-Labels, anonyme `-`/`+`-Labels, `+makro`-Aufrufe, `!zone`, `!for`, `!if`, `!do`, `!pseudopc`, `!convtab`, `!binary`, `!source`, `!to`. Für 68000-Quelltexte auch die Motorola-Datendirektiven ohne `!`: `dc.b`/`dc.w`/`dc.l`, `ds.b`/`ds.w`/`ds.l anzahl[, füllwert]` und `even`.
KickAssembler	`.asm`	`.`-Direktiven, `label:`, `!multi:`-Labels, `{ }`-Bereiche und die ganze Skriptsprache: `.var`/`.const`/`.eval`, Funktionen, Makros, Pseudobefehle, Listen, Hashtabellen, Strukturen, `.enum`, die Mathe-Bibliothek, Vektoren und Matrizen, der Präprozessor `#import`/`#if`, `.namespace`, Segmente, `.file`, `.disk`, `LoadBinary`/`LoadSid`/`LoadPicture`, `.assert`.
rat	`.rat`	rats eigener Dialekt: CPU-neutral, Direktiven im Kick-Stil, Kommentare mit `;` und `//`, dazu `.org`, `.include`, `.incbin`, `.fn`, `.for i in a..b`, `.elif`, `dc.b`/`ds.w`, `@lokale` und `.lokale` Labels. Teil II beschreibt ihn vollständig.

4.1 Welchen Dialekt eine Datei hat

Der erste Treffer gilt:

- Eine Zeile `.syntax acme|kick|rat` (oder `!syntax`) am Dateianfang.
- `--syntax` auf der Kommandozeile (nur für die Hauptdatei).
- Die Endung: `.a` ist ACME, `.asm` ist KickAssembler, `.rat` ist rat.
- Der Dialekt der Datei, die sie einbindet.

Jede eingebundene Datei wird für sich erkannt; ein Projekt kann also Dialekte mischen: eine KickAssembler-Hauptdatei kann eine ACME-Bibliothek und eine rat-Datei mit Tabellen einbinden.

> **Hinweis.** Die Operanden der Befehle stehen immer in der Schreibweise der CPU (Zilog, Intel, Motorola, Sony, MOS, TI, Acorn), gleich in welchem Dialekt. Ein Z80-Befehl liest sich in einem ACME-, einem KickAssembler- und einem rat-Quelltext gleich.

4.2 Dasselbe Programm in drei Dialekten

ACME

```
!to "hello.prg", cbm
!cpu 6510
* = $c000
CHROUT = $ffd2
!zone main
start   ldx #0
.loop   lda .msg,x
        beq +
        jsr CHROUT
        inx
        bne .loop
+       rts
.msg    !pet "hello", 13, 0
```

KickAssembler

```
        *= $c000
.const CHROUT = $ffd2
start:  ldx #0
!loop:  lda msg,x
        beq !done+
        jsr CHROUT
        inx
        bne !loop-
!done:  rts
msg:    .encoding "petscii_mixed"
        .text "hello"
        .byte 13, 0
```

rat

```
.platform c64
.org $c000
CHROUT = $ffd2
start:  ldx #0
@loop:  lda msg,x
        beq @done
        jsr CHROUT
        inx
        bne @loop
@done:  rts
msg:    .petscii "hello"
        .byte 13, 0
```

5 Plattformen

Eine **Plattform** legt die Standard-CPU, die Standard-Startadresse, die erlaubten Ausgabeformate, die Textkodierung und die Plattform-Direktiven fest. Gewählt wird sie auf der Kommandozeile mit `-p` oder im Quelltext:

```
.platform c64        ; KickAssembler und rat
!platform nes        ; ACME
```

Was nur zu einer Plattform gehört, gibt es nur dort. `BasicUpstart`, `LoadSid`, `LoadPicture`, `.disk` und die Formate `prg`, `d64` und `crt` sind Commodore-Funktionen. Auf `nes`, `cpc` oder `amiga` sind sie ein Fehler, der die nötige Plattform nennt. KickAssembler- und ACME-Quelltexte beginnen auf `c64`, bestehende Quelltexte bauen also unverändert; rat-Quelltexte beginnen auf `generic`.

> **Hinweis.** Die Standard-Startadresse gilt für die Plattform, mit der ein Bau beginnt (aus `-p` oder dem Dialekt). Nach einer `.platform`-Zeile im Quelltext setzt man die Adresse selbst mit `.org`, wie in allen Beispielen dieses Handbuchs.

5.1 Übersicht

Plattform	CPU	Formate (erstes = Standard)	Plattform-Direktiven
`c64`	6510	prg, d64, crt	`basicupstart`, `crt`, `disk`; `LoadSid()`, `LoadPicture()`
`c128`, `vic20`, `pet`, `plus4`	6502	prg, d64	`basicupstart`, `disk`; `LoadSid()`, `LoadPicture()`
`mega65`	45gs02	prg, d64	`basicupstart` (BASIC 65 `BANK 0:SYS`), `disk`; `LoadSid()`, `LoadPicture()`
`atari8`	6502	xex	`xex.run`, `xex.init`
`atari2600`	6507	a26	`a26.bank n`
`nes`	2a03	nes	`ines mapper= prg= chr= mirroring=`, `nes.chr { }`, `nes.bank n`
`snes`	65816	sfc	`snes.header title= map=lorom|hirom|exhirom`, `snes.spc [addr] { }`
`cpc`	z80	amsdos	`amsdos name= exec=`
`zx81`	z80	p	–
`spectrum`	z80	tap, sna	`tap name= exec= loader=`, `sna exec= sp= border= im= ei=`
`msx`, `msx2`	z80	bin (Modul-ROM), cas	`msx.header init`, `cas name= exec=`
`trs80`	z80	cas, cmd	`cas name= exec=`, `cmd exec`
`coleco`	z80	bin	`coleco.header start`
`imsai`	8080	bin	–
`bbc`, `bbcmaster`	6502, 65c02	ssd	`ssd name= exec= title= boot=`
`apple2`, `apple2e`	6502, 65c02	dsk	`dsk name= volume= type= locked=`
`mac`	68000	bin	–
`amiga`, `amiga1200`	68000, 68020	hunk (mit RELOC32)	`amiga.section name, code|data|bss`
`vectrex`	6809	bin	`vectrex.header "TITEL"`
`ti99`	tms9900	bin (Modul-ROM ab $6000)	`ti99.header "TITEL", start`

Plattform	CPU	Formate (erstes = Standard)	Plattform-Direktiven
`archimedes`	arm2	bin	–
`generic`, `sim6502`	6502	bin	–

`bin`, `hex` (Intel HEX) und `srec` (Motorola-S-Records) gibt es auf jeder Plattform. `rat platforms` und `rat formats` zeigen die vollständigen Listen.

5.2 Plattform-Direktiven

Eine Plattform-Direktive schreibt man `.name` …, solange ihre Plattform aktiv ist, oder überall `.<plattform>.<name>` …. Die zweite Form funktioniert auch, wenn eine andere Plattform aktiv ist:

```
.platform msx2
.header init            ; dasselbe wie .msx.header init

.c64.basicupstart start
.xex.run start
```

Einige Direktiven haben einen eigenen kurzen Namensraum: `xex` (Atari 8-Bit), `a26` (Atari 2600), `ines` (NES), `spc` (SNES), `amsdos` (CPC), `hunk` (Amiga), `tap` und `sna` (Spectrum), `dsk` (Apple II), `ssd` (BBC) und `cmd` (TRS-80).

6 Maschine für Maschine

Ein kurzes Beispiel für jede Gruppe von Maschinen. Die Quelltexte stammen aus rats Testkorpus (`tests/corpus`); jeder baut so, wie er hier steht.

6.1 Commodore: C64, C128, VIC-20, PET, C16/Plus/4, MEGA65

`basicupstart` schreibt eine BASIC-Zeile `10 SYS adresse` an den Anfang des BASIC-Speichers; der Code geht dahinter weiter. Beim MEGA65 lautet die Zeile `10 BANK 0:SYS adresse` ab `$2001`. `LoadSid()` und `LoadPicture()` lesen SID-Musik und Bilder, `.disk` baut ein D64 mit mehreren Dateien, `.crt` setzt den Kopf eines C64-Moduls.

```
.platform mega65
.basicupstart start
.org $2100
start:  ldq $10             ; 45GS02: 32-Bit-Register Q
        lda ($10),z
        sta [$10],z         ; flacher 28-Bit-Zeiger
        rts
```

```
rat build intro.asm                     # PRG
rat build intro.asm -f d64              # D64 mit dem PRG als einziger Datei
```

6.2 Atari 8-Bit und Atari 2600

```
.platform atari8
.org $2000
start:  lda #$22
        sta $d01a
        jmp start
.xex.run start              ; RUNAD-Segment: Startadresse nach dem Laden
```

```
.platform atari2600
.org $f000
reset:  sei
        jmp reset
.org $fffc
        .word reset, reset
```

`.a26.bank n` beginnt eine 4-KB-Bank eines F8-, F6- oder F4-Moduls; das Format `a26` schreibt 2 KB, 4 KB oder die Größen mit Bankumschaltung.

6.3 NES und SNES

```
.platform nes
.ines mapper=0, prg=1, chr=1, mirroring=v
.org $c000
reset:  sei
        cld
        ldx #$ff
        txs
-       bit $2002
        bpl -
        jmp *
nmi:    rti
.org $fffa
        .word nmi, reset, nmi
.nes.chr {
        .fill 16, $ff
}
```

Beim SNES schreibt `.snes.header` den internen ROM-Kopf, die Prüfsumme wird berechnet. `.snes.spc` assembliert einen Block mit dem SPC700 für den Tonprozessor und legt ihn an der aktuellen ROM-Position ab:

```
.platform snes
.snes.header title="RAT TEST", map=lorom
.org $008000
reset:  sei
        clc
        xce
        rep #$30
        .a16
        .i16
        lda #$1234
        jml reset
.snes.spc $0200 {
        mov a, #$10
        mov $f2, a
spin:   bra spin
}
```

6.4 Z80-Maschinen: CPC, ZX81, Spectrum, ColecoVision, MSX, MSX2, TRS-80

```
; Amstrad CPC: Binärdatei mit AMSDOS-Kopf
.platform cpc
.amsdos name="HELLO.BIN", exec=start
.org $4000
start:  ld hl, msg
@loop:  ld a, (hl)
        or a
        ret z
        call $bb5a          ; TXT OUTPUT
        inc hl
        jr @loop
msg:    .text "Hello CPC!", 13, 10
        .byte 0
```

Der ZX Spectrum schreibt eine TAP-Datei (mit `exec=` steht ein BASIC-Lader davor) oder einen 48K-Snapshot (`sna`). Der ZX81 schreibt eine `.p`-Datei.

MSX und MSX2

`msx` ist der MSX1, `msx2` der MSX2, den mimic als Modell `msx2` emuliert (Videochip V9938, Memory Mapper, Sub-ROM, Uhr). Beide haben dieselbe CPU, denselben Modulkopf und dasselbe Kassettenformat; ein Quelltext wechselt also mit der Plattformzeile zwischen ihnen. Ein Modul, das den V9938 oder das Sub-ROM benutzt, gehört auf `msx2`.

```
; ein 16-KB-Modul ab $4000
.platform msx2
.org $4000
.header init                ; "AB", INIT-Adresse, 12 Nullbytes

CHPUT = $00a2

init:   ld hl, text
@loop:  ld a, (hl)
        or a
        jr z, @halt
        call CHPUT
        inc hl
        jr @loop
@halt:  di
        halt
text:   .asciiz "Hello from rat"
        .fill $8000 - *, $ff
```

```
rat build cart.rat                      # -> cart.bin
mimic-desktop --machine msx --model msx2 --cart cart.bin
```

Eine Kassettendatei für `BLOAD"CAS:",R` entsteht mit dem Format `cas`:

```
.platform msx2
.cas name="HELLO", exec=main
.org $c000
main:   jr main
```

TRS-80

```
; Model I: ein SYSTEM-Band (SYSTEM, HELLO, /)
.platform trs80
.cas name="HELLO", exec=start
.org $5200
start:  ld hl, text
        ld de, $3c00        ; Bildschirmspeicher
        ld bc, textend - text
        ldir
@wait:  jr @wait
text:   .text "HELLO FROM RAT"
textend:
```

Das Format `cmd` schreibt ein TRSDOS-Lademodul; `.cmd exec` setzt dessen Einsprung.

6.5 Acorn: BBC Micro, BBC Master, Archimedes

```
; ein SSD-Image, das mit Shift+BREAK startet
.platform bbc
.ssd title="RAT", boot=true, exec=start
OSWRCH = $ffee
.org $1900
start:  ldx #0
@loop:  lda text,x
        beq @done
        jsr OSWRCH
        inx
        bne @loop
@done:  rts
text:   .asciiz "Hello from rat"
```

`bbcmaster` wählt den 65C02. Der Archimedes benutzt den ARM2 mit Acorn-Mnemonics und schreibt rohen Code:

```
.platform archimedes
.org $8000
        mov r0, #1
        stmfd r13!, {r0-r3, r14}
        bl sub
        ldmfd r13!, {r0-r3, pc}^
sub:    movs pc, r14
```

6.6 Apple II und Macintosh

```
; Apple IIe enhanced (65C02): DOS-3.3-Diskette mit dem Programm HELLO
.platform apple2e
.dsk name="HELLO", volume=254
.org $2000
start:  ldx #0
loop:   lda msg,x
        beq done
        jsr $fded           ; COUT
        inx
        bra loop
done:   rts
.encoding "apple"
msg:    .text "HELLO APPLE II", 13
        .byte 0
```

Das Format `dsk` ist ein DOS-3.3-Image mit dem Programm als Binärdatei: ein DOS booten und das Programm mit `BRUN` starten. Die Plattform Macintosh schreibt rohen 68000-Code.

6.7 Amiga

Das Format `hunk` ist ein ausführbares Programm für AmigaDOS. `.amiga.section` beginnt einen Hunk; absolute Langwörter, die in einen Hunk zeigen, werden reloziert (RELOC32).

```
.platform amiga
start:  move.l  #msg, a0
        lea     sub(pc), a1
        move.w  #$4000, $dff09a
        moveq   #0, d0
.loop:  addq.l  #1, d0
        cmp.l   #100, d0
        bne.s   .loop
        jsr     sub
        rts
sub:    rts
.amiga.section data, data
msg:    dc.b "hello", 0
ptr:    dc.l start, msg
```

`amiga1200` wählt den 68020 mit seinen zusätzlichen Adressierungsarten und Bitfeld-Befehlen.

6.8 Vectrex, TI-99/4A, IMSAI 8080

```
.platform vectrex
.org 0
.vectrex.header "RAT TEST"
main:   jsr $f192
        lda #$7f
        ldx #table
        ldd ,x++
        lbra main
table:  .word $1234, main
```

```
; TI-99/4A: ein Modul-ROM ab $6000
.platform ti99
.org $6000
.header "RAT DEMO", start
start:  limi 0
        lwpi $8300
        li r1, text
loop:   movb *r1+, @$8c00
        jmp loop
text:   .text "HELLO FROM RAT"
        .even
```

```
.platform imsai
.org $0100
        mvi a, $2a
        out $ff             ; Lampen der Frontplatte
        hlt
```

7 Zusammenspiel mit mimic

rat und mimic sind füreinander gebaut: rat schreibt die Dateien, die mimic lädt, und die Label-Datei zeigt im Debugger die eigenen Namen.

Ausgabe von rat	In mimic laden mit
`prg`, `xex`, Spectrum-`sna`	`--prg DATEI`
`d64`, Apple-`dsk`, BBC-`ssd`	`--disk DATEI`
`crt`, `nes`, `sfc`, `a26`, Modul-`bin` (MSX, ColecoVision, Vectrex, TI-99/4A)	`--cart DATEI`
`tap`, `cas`, `p`	`--tape DATEI`

```
rat build game.asm --vice-labels game.vs
mimic-desktop --machine c64 --prg game.prg --debug
```

- `--vice-labels` schreibt Zeilen der Form `al C:xxxx .name`. mimics Debugger und VICE laden sie direkt.
- `.break` und `.watch` im Quelltext fügen dieser Datei Haltepunkte hinzu.
- `--listing` zeigt, welche Bytes jede Quellzeile erzeugt hat, mit den Adressen, die der Debugger anzeigt.

Das Benutzerhandbuch von mimic beschreibt die Optionen des Emulators, die Medien jeder Maschine und den Debugger.

8 Zwischen Dialekten umwandeln

`rat convert` liest eine Datei und schreibt dasselbe Programm in einem anderen Dialekt.

- Kommentare und Leerzeilen bleiben erhalten.
- ACME-Zonen, `.lokale`/`@cheap`-Labels und anonyme Labels werden auf die `{ }`-Bereiche und `!`-Multi-Labels von KickAssembler abgebildet, und zurück.
- Operatoren werden für das Ziel umgeschrieben: aus ACMEs `=`, `XOR` und `^` werden zum Beispiel `==`, `^` und `pow()`.
- Die Byte-Reihenfolge der Daten folgt der CPU.

Was das Ziel nicht ausdrücken kann, wird als Warnung gemeldet und als Kommentar `TODO(rat convert)` stehen gelassen. Listen, Strukturen und Funktionen von KickAssembler etwa lassen sich nicht nach ACME umwandeln. Die Tests wandeln jede Korpusdatei in jeden Dialekt um und prüfen, dass das Ergebnis dieselben Bytes ergibt.

```
rat convert old.a --to kick -o new.asm
rat convert new.asm --to rat -o new.rat
rat convert tables.rat --from rat --to acme
```

9 REST-API

`rat serve` startet einen HTTP-Server (Standard `127.0.0.1:8080`). Die API ist als OpenAPI unter `/openapi.json` beschrieben, Swagger UI liegt unter `/docs`.

Methode	Pfad	Zweck
POST	`/assemble`	Ein Projekt assemblieren: JSON oder multipart/form-data.
POST	`/disassemble`	Bytes in rat-Syntax disassemblieren.
POST	`/convert`	Einen Quelltext zwischen den Dialekten umwandeln.
GET	`/cpus`, `/platforms`, `/formats`	Die Listen der Kommandozeile, als JSON.
GET	`/health`	Lebenszeichen.

Eine Anfrage sieht nur die Dateien, die sie hochlädt, über ein Dateisystem im Speicher; der Server fasst seine eigene Platte nie an. Der Rumpf einer Anfrage ist auf 32 MB begrenzt, jeder Lauf auf 30 Sekunden.

```
curl -s localhost:8080/assemble -H 'content-type: application/json' \
  -d '{"files":{"main.asm":"BasicUpstart2(start)\nstart: inc $d020\n jmp start\n"},
       "main":"main.asm"}'
```

Die Antwort enthält `ok`, die Ausgaben (Base64), die Meldungen mit Datei, Zeile, Spalte und dem aufbereiteten Text, die Symbole, die Ausgabe von `.print`, das Listing und die VICE-Labels. Teil II nennt jedes Feld.

Teil II — Referenz

Der rat-Dialekt, Operandenschreibweise, CPUs, Plattformen, Ausgabeformate, Optionen der Kommandozeile, die REST-API und bekannte Lücken

10 Der rat-Dialekt

rats eigener Dialekt ist für jede CPU gedacht, nicht nur für den 6502. Man benutzt ihn mit der Endung `.rat` oder mit `.syntax rat` am Dateianfang. Er liest sich wie KickAssembler ohne die C64-Annahmen, dazu ein paar Annehmlichkeiten aus ca65, vasm und Devpac.

```
; Kommentare: `;` oder `//`, Blockkommentare /* ... */
.platform nes                 ; CPU, Formate, Direktiven
.ines mapper=0, prg=1, chr=1, mirroring=v

.org $c000
reset:  sei
        ldx #$ff
        txs
@wait:  bit $2002             ; @lokal / .lokal: lokal zum letzten globalen Label
        bpl @wait
-       jmp -                 ; anonyme Labels wie in ACME
nmi:    rti
.org $fffa
        .word nmi, reset, nmi
```

10.1 Labels und Symbole

Schreibweise	Bedeutung
`name:`	Label am Programmzähler
`@name:` / `.name:`	lokales Label, sichtbar bis zum nächsten globalen Label
`-` `--` `+` `++`	anonyme Labels, angesprochen als `-`, `+`, … (wie in ACME)
`name = ausdruck`	konstantes Label
`.const x = e`, `.var x = e`, `.set x = e`, `.label x = e`	Konstante, Variable (`.eval x = …` ändert sie), Label
`name: { … }`, `.scope name { … }`	benannter Bereich; innere Labels erreicht man als `name.innen`
`.namespace ns { … }`	Namensraum

10.2 Zahlen, Zeichenketten, Operatoren

- **Zahlen:** `$ff`, `0xff`, `%1010`, `0b1010`, `0o17`, `123`, `1.5`, `'a'`.
- **Zeichenketten:** `"..."` mit den Escapes `\n \t \" \\ \0 \$xx`.
- **Operatoren** (wie in C, schwächste Bindung zuerst): `?:`, dann `||`, `&&`, `|`, `^`, `&`, dann `== !=`, dann `< <= > >=`, dann `<< >> >>>`, dann `+ -`, dann `* / %`, dann `**`, dann die einstelligen `- ! ~`.
- **`<` und `>` am Anfang eines Ausdrucks** nehmen das untere und das obere Byte von allem, was folgt: `#<table+1` ist `lo(table+1)`.
- **`/`** ist eine Ganzzahldivision, wenn beide Seiten ganze Zahlen sind.
- **Programmzähler:** `*` oder `$`.
- **Listen und Index:** `[1, 2, 3]` baut eine Liste; `liste[i]` liest ein Element.
- **Mathematische Funktionen:** `lo hi bank`, `abs min max floor ceil round sqrt sin cos tan pow` und weitere.
- **Funktionen für Zeichenketten:** `toIntString toHexString toBinaryString`, `len`, `defined(name)`.
- **Konstruktoren:** `List() Hashtable() Vector() Matrix()`.
- **Methoden** wie in KickAssembler: `list.add() .get() .size()`, `hash.put() .get() .keys()`, `string.substring() .toUpperCase()` und weitere.

10.3 Daten

Direktive	Bedeutung
`.byte/.db`, `.word/.dw`, `.long/.dword`, `.quad`	in der Byte-Reihenfolge der CPU (`.endian le|be` überschreibt sie)
`.le16 .be16 .le24 .be24 .le32 .be32`	ausdrückliche Byte-Reihenfolge
`dc.b dc.w dc.l`, `ds.b ds.w ds.l n[, füllwert]`	im 68k-Stil
`.text "…"`, `.asciiz "…"`, `.petscii "…"`, `.screen "…"`	Text in der aktuellen Kodierung, mit Null am Ende, PETSCII, C64-Bildschirmcodes
`.encoding "name"`	Textkodierung (der Standard kommt von der Plattform): `ascii`, `petscii_mixed`, `petscii_upper`, `screencode_mixed`, `screencode_upper`, `atari_screen`, `zx81`, `apple` (ASCII mit gesetztem Bit 7, für `COUT` des Apple II)
`.hex 0123 abcd`	Hex-Bytes
`.fill n, ausdruck`	der Ausdruck darf `i` benutzen, den Index (wie in KickAssembler)
`.lohifill n, ausdruck`	Tabellen der unteren und oberen Bytes, erreichbar als `label.lo` / `label.hi`
`.res n` / `.ds n`	n Nullbytes
`.align n[, füllwert]`, `.even`	Auffüllen
`.incbin "datei"[, offset[, länge]]`	Binärdatei einbinden

10.4 Programmzähler und Speicher

Direktive	Bedeutung
`.org adr`, `*= adr`, `.pc = adr "Blockname"`	den Programmzähler setzen (beginnt einen neuen Speicherblock)
`.pseudopc adr { … }` oder `.pseudopc adr … .realpc`	für eine andere Adresse assemblieren
`.segmentdef`, `.segment`, `.segmentout`, `.file`, `.memblock`	KickAssembler-Segmente
`.output "datei"[, format]`	Hauptausgabedatei und Format

10.5 Ablaufsteuerung und Wiederverwendung

```
.if x > 3 { … } .elif x { … } .else { … }
.ifdef NAME { … } .else { … }
.for i in 0..8 { .byte i * 2 }          ; ohne die 8; 0..=8 mit; `step n`
.for v in [1, 5, 9] { .byte v }
.while n < 10 { .eval n = n + 1 }
.repeat 4 { nop }
.macro poke(addr, value = 0) { lda #value
                               sta addr }
poke($d020, 1)        ; oder: poke $d020, 1
.fn double(x) = x * 2
.fn clamp(x) {
    .if x > 255 { .return 255 }
    .return x
}
.pseudocommand mov src : dst { lda src
                               sta dst }
mov #1 : $d020
.struct Point {x, y}
.enum {RED, GREEN = 5, BLUE}
#define DEBUG
#if DEBUG
  …
#endif
```

> **Hinweis.** `;` beginnt in rat einen Kommentar; Schleifen im C-Stil `for (…; …; …)` lassen sich deshalb nicht schreiben. Stattdessen `.for i in a..b` oder `.while` mit `.eval` benutzen.

10.6 Direktiven für CPU, Plattform und Ziel

Direktive	Bedeutung
`.cpu name`	die CPU wechseln: `6502`, `6502ni`, `6510`, `2a03`, `6507`, `65c02`, `45gs02`, `65816`, `spc700`, `z80`, `8080`, `6809`, `68000`, `68020`, `tms9900`, `arm2`
`.platform name [{ … }]`	die Plattform wechseln; wählt auch deren CPU
`.a8 .a16 .i8 .i16`	Registerbreiten des 65816
`.setdp seite`	Direct Page des 6809; angenommenes D-Register des 65816
`.endian le|be`	Byte-Reihenfolge der Daten
`.include "datei"`, `.once`, `.eof`	Quelldateien
`.print`, `.warn`, `.error`, `.errorif c, text`, `.assert bedingung[, text]`, `.asserterror`	Meldungen und Prüfungen
`.break`, `.watch`	Haltepunkte für die VICE-Label-Datei
`.syntax rat`	Dialektwahl (erste Zeile)

11 Operandenschreibweise je CPU

Operanden stehen immer in der Schreibweise der CPU, in jedem Dialekt.

CPU	Beispiele
65xx	`lda #<x`, `lda (zp),y`, `sta $1234,x`. Eine Größe erzwingen mit `lda.z` / `lda.a` (ACME: `lda+1` / `lda+2`)
45GS02	`lda ($10),z`, `sta [$10],z` (flach), `ldq $1234`, `adcq ($10)`, `lda ($10,sp),y`, `bbr3 $10,label`, `lbne label` (16 Bit; `bne` wächst von selbst), `phw #$1234`, `neg a`, `map`, `eom`
65816	`lda [dp],y`, `lda $12,s`, `lda ($12,s),y`, `jml $123456`, `mvn $7e,$00`, `lda.l`, `pea #x`
SPC700	`mov a,!abs+x`, `mov a,[dp+x]`, `mov a,[dp]+y`, `mov (x)+,a`, `set1 dp.3`, `mov1 c,abs.5`, `bbs dp.2,label`, `mov ziel,quelle`
Z80	`ld a,(ix+5)`, `ld (iy-2),$12`, `res 3,(ix+1),b`, `out (c),0`, `ex af,af'`, `ixh`/`ixl`, `sll`
8080	`mvi a,$2a`, `lxi h,table`, `mov m,a`, `push psw`, `jnz loop`, `rst 7`, `in $ff`
TMS9900	`li r1,$1234`, `mov *r1+,@$8c00`, `movb @table(r2),r3`, `ai r1,2`, `sla r3,4`, `jne loop`, `blwp @vector`, `rt` (Zahlen in rat-Schreibweise: `$8c00`, nicht `>8C00`)
ARM2	`addeqs r0,r1,r2,lsl #2`, `ldrb r0,[r1],#1`, `ldr r0,[r1,-r2]!`, `stmfd r13!,{r0-r3,r14}`, `ldmfd sp!,{pc}^`, `teqp pc,#0`, `bl label`, `swi $11`, `adr r0,label`, `ldr r0,label`; ein Direktwert, der sich nicht kodieren lässt, wird als MVN/BIC/SUB/CMN versucht
6809	`lda <$20` (direkt), `lda >$1234` (erweitert), `ldd ,x++`, `lda [,y]`, `leax 10,x`, `ldy label,pcr`, `pshs a,b,x`
68000/68020	`move.l #1,d0`, `lea 4(a0,d1.w*2),a1`, `(d16,pc)`, `movem.l d0-d7/a0-a6,-(sp)`, `bra.s`, `$dff180.l`, `([bd,a0],d1.w*4,od)`, `bfextu d0{4:8},d1`

12 CPUs

Die Kennungen gelten für `--cpu`, `.cpu` (KickAssembler, rat) und `!cpu` (ACME). Aliasnamen werden in jeder Groß- und Kleinschreibung angenommen, mit oder ohne `-` und führendes `_`. `rat cpus` zeigt diese Liste.

Kennung	Byte-Reihenf.	Adress-Bits	CPU	Aliasnamen
`6502`	little	16	MOS-6502-Familie mit undokumentierten Opcodes (6502, 8502, 7501/8501, SALLY)	`nmos6502`, `6502x`, `8502`, `7501`, `8501`, `sally`
`6502ni`	little	16	MOS 6502, nur dokumentierte Opcodes	`6502noillegals`
`6510`	little	16	MOS 6510 (C64) mit undokumentierten Opcodes	`nmos6510`
`2a03`	little	16	Ricoh 2A03/2A07 (NES), ohne Dezimalmodus	`2a07`, `nes`
`6507`	little	13	MOS 6507 (Atari 2600), 13-Bit-Adressbus	–
`65c02`	little	16	CMOS 65C02 (Apple IIe enhanced), ohne Rockwell-Bitbefehle	`65sc02`, `w65c02`, `g65sc02`, `65sc12`
`45gs02`	little	28	MEGA65 45GS02 (CSG 4510 / 65CE02 mit Q-Befehlen und flachen Zeigern)	`4510`, `65ce02`, `4502`, `m65`
`65816`	little	24	WDC 65C816 (SNES 5A22); Registerbreiten mit `.a8`/`.a16`/`.i8`/`.i16`	`65c816`, `w65c816`, `5a22`
`spc700`	little	16	Sony SPC700 (SNES-Ton), Sony-Schreibweise, `!` erzwingt absolut	`spc`
`z80`	little	16	Zilog Z80 mit undokumentierten Befehlen (IXH/IXL, SLL, DD-CB-Kopien)	`z80undoc`
`8080`	little	16	Intel 8080 (IMSAI 8080), Intel-Mnemonics	`i8080`, `8080a`
`6809`	big	16	Motorola 6809 / 68A09 (Vectrex); `setdp` für die Direct Page	`68a09`, `mc6809`
`68000`	big	24	Motorola 68000 (Amiga 500/1000/2000, Macintosh)	`68k`, `mc68000`
`68020`	big	24	Motorola 68020/68EC020 (Amiga 1200)	`68ec020`, `mc68020`
`tms9900`	big	16	Texas Instruments TMS9900 (TI-99/4A)	`9900`, `tms9900a`
`arm2`	little	26	ARM2 (Acorn Archimedes), Acorn-Mnemonics	`arm`, `arm26`, `arm3`

13 Plattformen

Kennungen und Aliasnamen gelten für `-p`, `.platform` und `!platform`. Das erste Format ist der Standard; `bin`, `hex` und `srec` sind überall erlaubt. `rat platforms` zeigt diese Liste mit den Direktiven.

Kennung	Aliasnamen	Maschine	CPU	Start	Formate	Kodierung
`generic`	`none`, `raw`	keine Plattform	6502	–	bin	ascii
`c64`	`commodore64`, `c-64`	Commodore 64	6510	$0801	prg, d64, crt	petscii_mixed
`c128`	`commodore128`	Commodore 128 (8502-Seite)	6502	$1C01	prg, d64	petscii_mixed
`vic20`	`vic-20`, `vc20`	Commodore VIC-20 (ohne Erweiterung)	6502	$1001	prg, d64	petscii_mixed
`pet`	`cbm`, `pet2001`	Commodore PET	6502	$0401	prg, d64	petscii_mixed
`plus4`	`c16`, `ted`, `264`, `c116`	Commodore 16 / Plus/4	6502	$1001	prg, d64	petscii_mixed
`mega65`	`m65`, `c65`	MEGA65	45gs02	$2001	prg, d64	petscii_mixed
`atari8`	`atari800`, `atarixl`, `atarixe`, `a8`, `atari`	Atari 400/800/XL/XE	6502	$2000	xex	ascii
`atari2600`	`vcs`, `a2600`, `2600`	Atari 2600 (VCS)	6507	$F000	a26	ascii
`nes`	`famicom`	Nintendo Entertainment System	2a03	$8000	nes	ascii
`snes`	`superfamicom`, `sfc`	Super Nintendo	65816	$008000	sfc	ascii
`cpc`	`amstrad`, `cpc464`, `cpc664`, `cpc6128`	Amstrad CPC	z80	$4000	amsdos	ascii
`zx81`	`ts1000`	Sinclair ZX81	z80	$4009	p	zx81
`spectrum`	`zxspectrum`, `zx`, `spectrum48`, `spectrum128`, `zx128`, `plus2`, `+2`, `speccy`	Sinclair ZX Spectrum 48K/128K/+2	z80	$8000	tap, sna	ascii
`coleco`	`colecovision`	ColecoVision	z80	$8000	bin	ascii
`msx`	`msx1`	MSX (MSX1)	z80	$4000	bin, cas	ascii
`msx2`	`msx-2`, `v9938`	MSX2	z80	$4000	bin, cas	ascii
`trs80`	`trs-80`, `trs80m1`, `trs80m3`, `model1`, `model3`	Tandy TRS-80 Model I / III	z80	$5200	cas, cmd	ascii
`imsai`	`imsai8080`, `s100`	IMSAI 8080	8080	$0100	bin	ascii
`bbc`	`bbcmicro`, `bbcb`, `bbcbplus`, `beeb`	Acorn BBC Micro Model B / B+	6502	$1900	ssd	ascii
`bbcmaster`	`master`, `master128`	Acorn BBC Master 128 (65SC12)	65c02	$1900	ssd	ascii
`archimedes`	`arc`, `a305`, `a310`, `a440`, `a3000`, `riscos`	Acorn Archimedes A305/A310/A440/A3000	arm2	$8000	bin	ascii
`apple2`	`appleii`, `apple`, `apple2plus`, `iiplus`, `a2`	Apple II / II+ / IIe	6502	$2000	dsk	ascii
`apple2e`	`apple2e_enhanced`, `iie`, `apple2c`, `iic`	Apple IIe enhanced (65C02)	65c02	$2000	dsk	ascii
`mac`	`macintosh`, `mac128`, `mac512`, `macplus`, `macse`, `mac128k`, `mac512k`	Apple Macintosh 128K/512K/Plus/SE	68000	0	bin	ascii

Kennung	Aliasnamen	Maschine	CPU	Start	Formate	Kodierung
`amiga`	`a500`, `a1000`, `a2000`, `a500plus`, `ocs`, `ecs`	Amiga 500/1000/2000	68000	0	hunk	ascii
`amiga1200`	`a1200`, `aga`	Amiga 1200	68020	0	hunk	ascii
`vectrex`	`gce`	Vectrex	6809	$0000	bin	ascii
`ti99`	`ti994a`, `ti-99`, `ti-99/4a`, `ti99/4a`	Texas Instruments TI-99/4A	tms9900	$6000	bin	ascii
`sim6502`	`sim`	mimics Testmaschine sim6502	6502	$0400	bin	ascii

13.1 Plattform-Direktiven und -Funktionen

Plattformen	Direktive	Bedeutung
`c64`	`.basicupstart adresse`	BASIC-Zeile `10 SYS adresse` ab $0801 (KickAssembler `BasicUpstart2`)
`c128`, `vic20`, `pet`, `plus4`	`.basicupstart adresse`	BASIC-Zeile `10 SYS adresse` am BASIC-Anfang; der Code geht dahinter weiter
`mega65`	`.basicupstart adresse`	BASIC-65-Zeile `10 BANK 0:SYS adresse` ab $2001
`c64`	`.crt type=, name=, exrom=, game=`	Kopf eines CRT-Moduls für das Format `crt`
alle Commodore, `mega65`	`.disk [filename=, name=, id=] { [name=, type=, segments=] ... }`	ein D64-Image bauen (KickAssembler `.disk`)
alle Commodore, `mega65`	`LoadSid()`, `LoadPicture()`, `BasicUpstart`, `BasicUpstart2`	Funktionen und Makros des Commodore-Vorspanns
`atari8`	`.run adresse`	RUNAD-Segment: Startadresse nach dem Laden
`atari8`	`.init adresse`	INITAD-Segment nach dem aktuellen Segment
`atari2600`	`.bank n [, adresse]`	4K-Bank n beginnen (Bankumschaltung F8/F6/F4), Standardadresse $F000
`nes`	`.ines mapper=, prg=, chr=, mirroring=h|v, battery=`	iNES-Kopf; prg in 16K-Einheiten, chr in 8K-Einheiten
`nes`	`.chr { ... }`	den Block ins CHR-ROM assemblieren (ab Adresse 0)
`nes`	`.bank n [, adresse]`	16K-PRG-Bank n beginnen
`snes`	`.header title=, map=lorom|hirom|exhirom, fast=, type=, sram=, region=, version=`	interner ROM-Kopf; die Prüfsumme wird berechnet
`snes`	`.spc [adresse] { ... }`	SPC700-Code, assembliert für `adresse` (Standard $0200), abgelegt an der aktuellen ROM-Position
`cpc`	`.amsdos name=, exec=`	AMSDOS-Kopf: Dateiname und Einsprung
`spectrum`	`.tap name=, exec=, loader=`	TAP-Ausgabe: Name des CODE-Blocks; mit `exec=` (oder `loader=true`) steht ein BASIC-Lader CLEAR / LOAD "" CODE / RANDOMIZE USR davor
`spectrum`	`.sna exec=, sp=, border=, im=, ei=`	SNA-Ausgabe: Einsprung (Standard: niedrigste Adresse), Stapel, Rahmenfarbe, Interrupt-Modus
`coleco`	`.header start [, titel]`	Modulkopf ab $8000 ($AA55, Zeiger, Startadresse)
`msx`, `msx2`	`.header init`	Modulkopf (`AB`, INIT-Adresse, 12 Nullbytes)
`msx`, `msx2`	`.cas name=, exec=`	CAS-Ausgabe: Name und Startadresse der BLOAD-Datei
`trs80`	`.cas name=, exec=`	CAS-Ausgabe: Name und Einsprung des SYSTEM-Bands

Plattformen	Direktive	Bedeutung
`trs80`	`.cmd exec`	CMD-Ausgabe: Einsprung des Lademoduls
`bbc`, `bbcmaster`	`.ssd name=, exec=, title=, boot=`	SSD-Ausgabe: DFS-Dateiname, Ausführungsadresse, Diskettentitel; `boot=true` nennt die Datei `!BOOT` und setzt `*OPT 4,2` (Shift+BREAK startet sie)
`apple2`, `apple2e`	`.dsk name=, volume=, type=, locked=`	DSK-Ausgabe: Katalogname, Volume-Nummer (254), DOS-Dateityp ($04 binär)
`amiga`, `amiga1200`	`.section name, code|data|bss`	einen Hunk beginnen; absolute Langwörter werden reloziert (RELOC32)
`vectrex`	`.header titel [, musik]`	Modulkopf (Copyright `g GCE`, Musik, Titel)
`ti99`	`.header titel, start`	Modulkopf ab $6000 mit einem Programmeintrag (Name in der Auswahlliste, Startadresse)

14 Ausgabeformate

Ein Format wählt man mit `-f`, mit `.output "datei", format` oder über den Standard der Plattform. `rat formats` zeigt diese Liste.

Format	Endung	Inhalt	Plattformen
`bin`	`.bin`	rohes Binärabbild, von der niedrigsten bis zur höchsten geschriebenen Adresse	alle
`hex`	`.hex`	Intel HEX	alle
`srec`	`.s19`	Motorola-S-Records	alle
`prg`	`.prg`	Commodore-PRG (Ladeadresse + Daten)	c64, c128, vic20, pet, plus4, mega65
`d64`	`.d64`	1541-Disketten-Image	c64, c128, vic20, pet, plus4, mega65
`crt`	`.crt`	C64-CRT-Modul	c64
`xex`	`.xex`	Atari-8-Bit-XEX (Segmente, RUNAD/INITAD)	atari8
`a26`	`.a26`	Atari-2600-Modul (2K/4K, Bänke F8/F6/F4)	atari2600
`nes`	`.nes`	iNES-ROM (Kopf + PRG + CHR)	nes
`sfc`	`.sfc`	SNES-ROM (LoROM/HiROM/ExHiROM, Prüfsumme gesetzt)	snes
`amsdos`	`.bin`	Amstrad-CPC-Binärdatei mit AMSDOS-Kopf	cpc
`p`	`.p`	ZX81-.P-Datei (Speicher ab $4009)	zx81
`tap`	`.tap`	ZX-Spectrum-Band (CODE-Block, wahlweise BASIC-Lader)	spectrum
`sna`	`.sna`	ZX-Spectrum-48K-Snapshot (beginnt am Einsprung)	spectrum
`cas`	`.cas`	Kassetten-Image: MSX-BLOAD-Datei, TRS-80-SYSTEM-Band	msx, msx2, trs80
`cmd`	`.cmd`	TRS-80-/CMD-Lademodul	trs80
`dsk`	`.dsk`	Apple-II-DOS-3.3-Image mit dem Programm als Binärdatei	apple2, apple2e
`ssd`	`.ssd`	BBC-Micro-DFS-Image (40 Spuren) mit dem Programm als Datei	bbc, bbcmaster
`hunk`	`.exe`	Amiga-Hunk-Programm (mit RELOC32)	amiga, amiga1200

15 Referenz der Kommandozeile

15.1 rat build QUELLE

Option	Bedeutung
`-o, --output DATEI`	Ausgabedatei (Standard: Name des Quelltexts mit der Endung des Formats).
`--cpu CPU`	CPU; überschreibt den Standard der Plattform.
`-p, --platform PLATTFORM`	Zielplattform.
`-s, --syntax DIALEKT`	Dialekt der Hauptdatei: `acme`, `kick`, `rat`.
`-f, --format FORMAT`	Ausgabeformat.
`-D, --define NAME[=WERT]`	Ein Symbol definieren (auch in `cmdLineVars`); mehrfach erlaubt.
`-I, --include ORDNER`	Include-Verzeichnis; mehrfach erlaubt.
`--listing DATEI`	Ein Listing schreiben.
`--vice-labels DATEI`	VICE-Labels schreiben (lesbar für mimics Debugger).
`--sym DATEI`	Eine KickAssembler-`.sym`-Datei schreiben.
`--labels DATEI`	Eine ACME-Label-Liste schreiben.
`--passes N`	Höchstzahl der Durchläufe (Standard 20).
`--json`	Eine Zusammenfassung von Ausgaben und Symbolen als JSON ausgeben.

15.2 rat disasm DATEI

Option	Bedeutung
`--cpu CPU`	CPU (Pflicht).
`--org ADR`	Ladeadresse (Standard: aus einem PRG-Kopf, sonst 0).
`--skip N`	Bytes am Anfang überspringen (Standard 0).
`--prg`	Die Datei als PRG behandeln (2 Bytes Ladeadresse).
`--a16, --i16`	65816: 16-Bit-Akkumulator, 16-Bit-Indexregister.

15.3 rat convert DATEI

Option	Bedeutung
`--to DIALEKT`	Zieldialekt: `acme`, `kick`, `rat` (Pflicht).
`--from DIALEKT`	Quelldialekt (Standard: erkannt).
`-o, --output DATEI`	Ausgabedatei (Standard: Standardausgabe).

15.4 rat serve

Option	Bedeutung
`--bind ADR`	Adresse und Port (Standard `127.0.0.1:8080`).

16 Referenz der REST-API

16.1 POST /assemble

JSON-Rumpf oder multipart/form-data mit Dateiteilen und Textfeldern.

Feld	Bedeutung
`files`	Dateien nach Pfad: Klartext oder `{"base64": "..."}` für Binärdateien. Beispiel: `{"main.asm": "...", "gfx/logo.bin": {"base64": "..."}}`.
`main`	Pfad des Hauptquelltexts in `files`.
`cpu`, `platform`, `syntax`, `format`	Wahlweise; wie auf der Kommandozeile.
`defines`	Wahlweise ein Objekt mit `-D`-Definitionen, Name zu Wert.
`listing`	Wahlweise; `true` liefert das Listing und die Quellzeile zu jeder Adresse mit.

In einem Multipart-Formular ist der Dateiname eines Teils sein Pfad; die Textfelder heißen `main`, `cpu`, `platform`, `syntax`, `format`, `listing` und `define` (`NAME=WERT`, mehrfach erlaubt).

Feld der Antwort	Bedeutung
`ok`, `passes`, `cpu`, `platform`	Ergebnis, Zahl der Durchläufe, benutzte CPU und Plattform.
`outputs`	Liste von `{name, format, size, base64}`.
`diagnostics`	Liste von `{severity, message, file, line, column, rendered}`; `severity` ist error, warning oder info, `rendered` der Text mit der Quellzeile.
`symbols`	Liste von `{name, value, label}`.
`prints`	Die Ausgabe von `.print`.
`listing`, `lines`	Mit `listing`: der Text des Listings und `{file, line, addr, size}` zu jeder Zeile.
`vice_labels`	VICE-Monitor-Labels.

16.2 POST /disassemble

Feld	Bedeutung
`cpu`	Kennung der CPU.
`org`	Ladeadresse (Standard 0).
`base64`	Die Bytes.
`a16`, `i16`	65816: 16-Bit-Akkumulator, 16-Bit-Indexregister.

Die Antwort enthält `lines` (`{addr, bytes, text}`) und `source`, den Quelltext im rat-Dialekt.

16.3 POST /convert

Feld	Bedeutung
`source`	Der Quelltext.
`name`	Wahlweise ein Dateiname; seine Endung hilft, den Dialekt zu erkennen.
`from`	Wahlweise der Quelldialekt.
`to`	Zieldialekt.

Die Antwort enthält `source` und `warnings`.

16.4 GET /cpus, /platforms, /formats, /health

`/cpus` nennt Kennung, Aliasnamen, Beschreibung, Byte-Reihenfolge, Adress-Bits und die Mnemonics jeder CPU; `/platforms` Kennung, Aliasnamen, Name, CPU, Formate, Standard-Startadresse, Direktiven und Funktionen; `/formats` Name, Endung, Beschreibung und Plattformen (`*` heißt alle). Eine Fehlerantwort ist `{"error": "..."}`.

17 Bekannte Lücken

- KickAssembler: `.plugin` (Java), `.modify`/`.filemodify` und Segment-Modifier, `createFile` sowie Label-Arrays von Schleifen (`loop[i].label`) werden nicht unterstützt. Die CPU `dtv` fehlt, weil mimic sie nicht emuliert.
- `65c02` hat keine Rockwell/WDC-Bitbefehle (`RMB`, `SMB`, `BBR`, `BBS`, `STP`, `WAI`), wie mimics 65C02 (der NCR/GTE-Baustein des Apple IIe).
- Es gibt keine eigene CPU 4510 oder 65CE02: `!cpu 65ce02`, `4510` und `4502` wählen den 45GS02, der beide umfasst.
- MSX und MSX2 teilen sich eine Plattformdefinition; es gibt keine MSX2-eigenen Direktiven und keinen Schreiber für Disketten-Images.
- Archimedes, TI-99/4A und IMSAI geben rohen Code aus (`bin`, `hex`, `srec`): es gibt keinen Schreiber für RISC-OS-ADFS-Images, TI-Disketten oder Kassetten.
- Der Macintosh gibt rohen Code aus; es gibt keinen Schreiber für Resource Forks oder MFS-Disketten.
- Amiga: es gibt keinen Schreiber für ADF- oder Festplatten-Images (HDF); das Hunk-Programm kommt mit anderen Werkzeugen auf eine Diskette.
- mimics eigener Logophor 16 (CPU ADA) ist kein Ziel von rat; er hat seinen Assembler in mimic.
- Die Umwandlung von KickAssembler-Skriptcode (Listen, Strukturen, Funktionen, Pseudobefehle, Segmente) nach ACME hinterlässt Kommentare `TODO(rat convert)`.

18 Wie rat arbeitet

```
crates/
  rat-core      Spans, Meldungen, Tokens, einstellbarer Lexer, Anweisungs-IR
  rat-cpu       CPU-Backends (Encoder + Disassembler je CPU)
  rat-syntax    Parser für ACME und Kick/rat -> gemeinsame IR
  rat-platform  Plattformverzeichnis, Commodore-Vorspänne, SID/Bilder laden
  rat-output    prg, xex, nes, sfc, a26, p, amsdos, hunk, crt, d64, tap, sna,
                dsk, ssd, cas, cmd, hex, srec, Symboldateien
  rat-asm       die Mehrpass-Maschine, Skriptwerte, Umwandler
  rat-server    REST-API
  rat-cli       das Programm `rat`
tests/          Rundläufe gegen mimic, Korpus mit Soll-Ausgaben
tools/          der Erzeuger dieses Handbuchs
```

18.1 Durchläufe

Die Maschine führt das Programm einmal je Durchlauf aus. Ein Vorwärtsbezug benutzt den Wert aus dem vorigen Durchlauf. Operandengrößen wie Zero Page gegen absolut oder kurze gegen lange Sprünge können nur wachsen, die Durchläufe konvergieren also. Das Assemblieren endet, wenn sich kein Symbol mehr ändert. Kommen die Werte nie zur Ruhe, werden die undefinierten Symbole gemeldet.

18.2 Relozierung beim Amiga

Hunk-Programme werden reloziert, indem noch einmal mit einem verschobenen Hunk assembliert wird: Langwörter, die sich mitbewegen, werden zu RELOC32-Einträgen.

18.3 Tests

```
cargo test --workspace --release
```

- **Rundläufe gegen mimic.** Jeder Opcode von 6502, 65C02, 45GS02, 65816, SPC700, 8080, Z80 und 6809, jedes Opcode-Wort von 68000 und TMS9900, ein Drittel des 68020-Raums und jede Kombination aus Bedingung, Klasse und Flags des ARM2 wird mit mimics Disassembler disassembliert, mit rat assembliert, wieder disassembliert und verglichen.
- **Korpus.** Quelltexte für die Plattformen werden gegen Soll-Ausgaben assembliert und in jeden Dialekt umgewandelt.
- **Medien.** Band- und Disketten-Images werden mit mimics Medienlesern zurückgelesen.
- **REST.** Assemblieren über JSON und Multipart, Disassemblieren, Umwandeln und das OpenAPI-Dokument.

18.4 Dieses Handbuch bauen

```
pip install reportlab
python tools/user_guide.py              # -> docs/rat-user-guide-en.pdf, -de.pdf
```